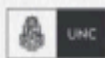


CERTIFICACIÓN UNIVERSITARIA



Manual para el alumno

Práctica Profesional VI



**WE WANT
SKILLS!**

mundosE

Manual para el alumno: Práctica Profesional VI.....	2
Objetivos del Laboratorio.....	2
Prerrequisitos.....	2
 Parte 1: Preparación del Sistema e Instalación de Herramientas.....	3
Paso 1.1: Actualización del sistema.....	3
Paso 1.2: Instalación de herramientas básicas.....	3
Paso 1.3: Instalación de Terraform (Método oficial).....	3
Paso 1.4: Instalación de OpenTofu (Método oficial).....	5
Paso 1.5: Instalación de AWS CLI.....	6
Paso 1.6: Configuración de AWS CLI.....	6
 Parte 2: Creación de la configuración.....	7
Paso 2.1: Crear el directorio del proyecto.....	7
Paso 2.2: Crear el archivo de variables.....	8
Paso 2.3: Crear el archivo principal.....	9
Paso 2.4: Crear el archivo de outputs.....	13
 Parte 3: Despliegue con Terraform.....	15
Paso 3.1: Inicialización.....	15
Paso 3.2: Validación.....	15
Paso 3.3: Planificación.....	15
Paso 3.4: Aplicación.....	16
Paso 3.5: Guardar la clave SSH.....	16
 Parte 4: Verificación del despliegue.....	17
Paso 4.1: Probar la aplicación.....	17
Si recibes "Connection refused" - Pasos de Validación.....	17
Paso 4.2: Verificar desde el navegador.....	19
Paso 4.3: Acceso SSH.....	19
 Parte 5: Migración a OpenTofu.....	19
Paso 5.1: Crear directorio para OpenTofu.....	19
Paso 5.2: Copiar configuración.....	20
Paso 5.3: Inicializar OpenTofu.....	20
Paso 5.4: Importar estado existente.....	20
Paso 5.5: Verificar con OpenTofu.....	20
 Parte 6: Limpieza.....	21
Paso 6.1: Destruir recursos.....	21

Este manual es un complemento de los encuentros sincrónicos de cada módulo. En este encontrarás un resumen de cada tema tratado en el encuentro, así como también, recursos adicionales para poder profundizar sobre los conceptos vistos.

Manual para el alumno: Práctica Profesional VI

Objetivos del Laboratorio

- Implementar Infrastructure as Code usando Terraform y OpenTofu
- Desplegar infraestructura en AWS de forma automatizada
- Gestionar el estado de Terraform usando S3 y DynamoDB
- Demostrar la intercambiabilidad entre Terraform y OpenTofu
- Comprender las mejores prácticas de IaC

Prerrequisitos

- Ubuntu 22.04 o superior con conexión a internet.
- Cuenta AWS con credenciales de acceso programático (Access Key ID + Secret Access Key).
- Conocimientos básicos de línea de comandos de Linux.
- Permisos de sudo en el sistema Ubuntu.

Nota: En este laboratorio instalaremos todas las herramientas necesarias (Terraform, OpenTofu, AWS CLI) desde cero.

Parte 1: Preparación del Sistema e Instalación de Herramientas

Paso 1.1: Actualización del sistema

Primero, actualizamos el sistema Ubuntu:

```
sudo apt update && sudo apt upgrade -y
```

Paso 1.2: Instalación de herramientas básicas

Instalamos las herramientas necesarias:

```
sudo apt install -y curl wget unzip git software-properties-common gnupg  
lsb-release
```

Salida esperada:

```
Leyendo lista de paquetes... Hecho  
Creando árbol de dependencias... Hecho  
Los siguientes paquetes se instalarán:  
  curl wget unzip git software-properties-common gnupg lsb-release  
0 actualizados, 7 nuevos se instalarán, 0 para eliminar y 0 no actualizados.  
Se necesita descargar 2,543 kB de archivos.  
Procesando disparadores para man-db (2.10.2-1) ...
```

 **Explicación:** Estas herramientas son necesarias para:

- ☐ **curl/wget:** Descargar archivos desde internet
- ☐ **unzip:** Extraer archivos comprimidos
- ☐ **git:** Control de versiones (buena práctica en IaC)
- ☐ **gnupg:** Verificar firmas digitales de paquetes
- ☐ **software-properties-common:** Gestionar repositorios adicionales

Paso 1.3: Instalación de Terraform (Método oficial)

Agregamos el repositorio oficial de HashiCorp:

Agregar la clave GPG oficial de HashiCorp

```
curl -fsSL https://apt.releases.hashicorp.com/gpg | sudo gpg --dearmor -o /usr/share/keyrings/hashicorp-archive-keyring.gpg
```

Agregar el repositorio oficial de HashiCorp

```
echo "deb [signed-by=/usr/share/keyrings/hashicorp-archive-keyring.gpg] https://apt.releases.hashicorp.com $(lsb_release -cs) main" | sudo tee /etc/apt/sources.list.d/hashicorp.list
```

Actualizar lista de paquetes e instalar Terraform

```
sudo apt update
```

```
sudo apt install -y terraform
```

Verificar la instalación:

```
terraform version
```

Agregar la clave GPG oficial de HashiCorp

```
curl -fsSL https://apt.releases.hashicorp.com/gpg | sudo gpg --dearmor -o /usr/share/keyrings/hashicorp-archive-keyring.gpg
```

Agregar el repositorio oficial de HashiCorp

```
echo "deb [signed-by=/usr/share/keyrings/hashicorp-archive-keyring.gpg] https://apt.releases.hashicorp.com $(lsb_release -cs) main" | sudo tee /etc/apt/sources.list.d/hashicorp.list
```

Actualizar lista de paquetes e instalar Terraform

```
sudo apt update
```

```
sudo apt install -y terraform
```

Verificar la instalación:

```
terraform version
```

Salida esperada:

```
Terraform v1.6.6  
on linux_amd64
```

💡 **Explicación:** Terraform es la herramienta principal de Infrastructure as Code que usaremos. La instalamos desde el repositorio oficial de HashiCorp para garantizar que tengamos una versión estable y con soporte.

Paso 1.4: Instalación de OpenTofu (Método oficial)

Instalamos OpenTofu usando el instalador oficial:

```
# Descargar el instalador oficial  
  
curl --proto '=https' --tlsv1.2 -fsSL https://get.opentofu.org/install-opentofu.sh -o  
install-opentofu.sh  
  
# Revisar el script antes de ejecutarlo (buena práctica de seguridad)  
  
cat install-opentofu.sh  
  
# Ejecutar el instalador  
  
sudo bash install-opentofu.sh --install-method deb  
  
# Limpiar archivo temporal  
  
rm install-opentofu.sh  
  
Verificar la instalación:  
  
tofu version
```

Salida esperada:

```
OpenTofu v1.8.4  
on linux_amd64
```

💡 **Explicación:** OpenTofu es un fork open-source de Terraform, 100% compatible. Lo instalamos para demostrar la intercambiabilidad entre herramientas de IaC y mostrar alternativas de código abierto.

Paso 1.5: Instalación de AWS CLI

Instalamos AWS CLI para interactuar con los servicios de AWS:

```
# Descargar e instalar AWS CLI v2

curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o "awscliv2.zip"

unzip awscliv2.zip

sudo ./aws/install

rm -rf awscliv2.zip aws/

Verificar la instalación:

aws --version
```

Salida esperada:

```
aws-cli/2.17.44 Python/3.11.6 Linux/6.2.0-1015-aws exe/x86_64.ubuntu.22
prompt/off
```

Paso 1.6: Configuración de AWS CLI

Configura tus credenciales de AWS:
aws configure

Te pedirá ingresar la siguiente información:

```
AWS Access Key ID [None]: AKIAIOSFODNN7EXAMPLE
AWS Secret Access Key [None]:
wJalrXUtnFEMI/K7MDENG/bPxrRfCYEXAMPLEKEY
Default region name [None]: us-east-1
```

```
Default output format [None]: json
```

 **Explicación:** Las credenciales AWS se obtienen desde la consola de AWS:

1. Inicia sesión en AWS Console
2. Ve a IAM → Users → Tu usuario → Security credentials
3. Crea "Access keys" para acceso programático
4. Copia el Access Key ID y Secret Access Key

 **Seguridad:** Nunca compartas estas credenciales. Mantenlas privadas y seguras.

Verificar que las credenciales funcionen:

```
aws sts get-caller-identity
```

Salida esperada:

```
{
  "UserId": "AIDACKCEVSQ6C2EXAMPLE",
  "Account": "123456789012",
  "Arn": "arn:aws:iam::123456789012:user/terraform-user"
}
```

 **Explicación:** Este comando confirma que tienes credenciales AWS válidas y funcionando. Sin esto, Terraform no podrá crear recursos en AWS.

Parte 2: Creación de la configuración

Paso 2.1: Crear el directorio del proyecto

```
mkdir -p laboratorio-terraform
cd laboratorio-terraform
pwd
```


Salida esperada:

```
/home/usuario/laboratorio-terraform
```

 **Explicación:** Creamos un directorio dedicado para nuestro proyecto de Infrastructure as Code. Es una buena práctica mantener cada proyecto en su propio directorio.

Paso 2.2: Crear el archivo de variables

```
Crea el archivo variables.tf:  
nano variables.tf
```

 **Explicación:** El archivo **variables.tf** define todas las variables que podemos personalizar en nuestro proyecto. Esto hace el código reutilizable y configurable.

Agrega el siguiente contenido en el editor nano:

```
variable "aws_region" {  
  description = "Región de AWS donde se van a crear todos los recursos"  
  type       = string  
  default    = "us-east-1"  
}  
  
variable "environment" {  
  description = "Ambiente de trabajo"  
  type       = string  
  default    = "laboratorio"  
}  
  
variable "instance_type" {  
  description = "Tipo de instancia EC2"  
  type       = string  
  default    = "t2.micro"  
}  
  
variable "ami_id" {  
  description = "ID de la imagen Ubuntu 22.04"  
  type       = string
```

```
default    = "ami-0866a3c8686eaeeba"
}
```

Guarda el archivo con **Ctrl+X**, luego **Y**, luego **Enter**.

Para verificar que se guardó correctamente:

```
cat variables.tf
```

Salida esperada:

```
variable "aws_region" {
  description = "Región de AWS donde se van a crear todos los recursos"
  type        = string
  default     = "us-east-1"
}
...
```

Paso 2.3: Crear el archivo principal

Crea el archivo **main.tf**:
`nano main.tf`

 **Explicación:** El archivo **main.tf** contiene la configuración principal de nuestra infraestructura. Aquí definimos todos los recursos que queremos crear en AWS.

Agrega el siguiente contenido en el editor nano:

```
nano main.tf
```

Agrega el siguiente contenido:

```
terraform {
  required_version = ">= 1.0"
  required_providers {
    aws = {
      source = "hashicorp/aws"
    }
  }
}
```

```

    version = "~> 5.0"
  }
  tls = {
    source = "hashicorp/tls"
    version = "~> 4.0"
  }
}
}
provider "aws" {
  region = var.aws_region
  default_tags {
    tags = {
      Proyecto   = "PP6-IaC"
      Ambiente   = var.environment
      Creado_Por = "Terraform"
      Laboratorio = "Infraestructura-como-Codigo"
    }
  }
}
# Crear una VPC
resource "aws_vpc" "pp6_vpc" {
  cidr_block      = "10.0.0.0/16"
  enable_dns_hostnames = true
  enable_dns_support = true
  tags = {
    Name = "pp6-vpc-laboratorio"
  }
}
# Crear una subnet pública
resource "aws_subnet" "pp6_public_subnet" {
  vpc_id            = aws_vpc.pp6_vpc.id
  cidr_block        = "10.0.1.0/24"
  availability_zone = "${var.aws_region}a"
  map_public_ip_on_launch = true
  tags = {
    Name = "pp6-subnet-publica"
  }
}
# Crear un Internet Gateway
resource "aws_internet_gateway" "pp6_igw" {
  vpc_id = aws_vpc.pp6_vpc.id
  tags = {
    Name = "pp6-internet-gateway"
  }
}

```

```

}

# Crear una tabla de ruteo
resource "aws_route_table" "pp6_public_rt" {
  vpc_id = aws_vpc.pp6_vpc.id
  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.pp6_igw.id
  }
  tags = {
    Name = "pp6-tabla-ruteo-publica"
  }
}

# Asociar la subnet con la tabla de ruteo
resource "aws_route_table_association" "pp6_public_rta" {
  subnet_id    = aws_subnet.pp6_public_subnet.id
  route_table_id = aws_route_table.pp6_public_rt.id
}

# Grupo de seguridad
resource "aws_security_group" "pp6_app_sg" {
  name_prefix = "pp6-app-"
  description = "Grupo de seguridad para la aplicacion PP6 laC"
  vpc_id      = aws_vpc.pp6_vpc.id
  ingress {
    description = "Acceso SSH"
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  ingress {
    description = "Aplicacion Web Python"
    from_port   = 3000
    to_port     = 3000
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
    description = "Todo el trafico saliente"
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
  }
}

```

```

    cidr_blocks = ["0.0.0.0/0"]
  }
  tags = {
    Name = "pp6-grupo-seguridad-app"
  }
}
# Generar clave SSH automáticamente
resource "tls_private_key" "pp6_key_pair" {
  algorithm = "RSA"
  rsa_bits  = 2048
}
resource "aws_key_pair" "pp6_key" {
  key_name   = "pp6-clave-ssh"
  public_key = tls_private_key.pp6_key_pair.public_key_openssh
  tags = {
    Name = "pp6-clave-ssh"
  }
}

# Instancia EC2
resource "aws_instance" "pp6_app" {
  ami                = var.ami_id
  instance_type      = var.instance_type
  key_name            = aws_key_pair.pp6_key.key_name
  vpc_security_group_ids = [aws_security_group.pp6_app_sg.id]
  subnet_id          = aws_subnet.pp6_public_subnet.id

  associate_public_ip_address = true
  user_data = base64encode(<<-EOF

#!/bin/bash
apt-get update -y
apt-get install -y python3 net-tools
mkdir -p /home/ubuntu/pp6-web
cd /home/ubuntu/pp6-web

cat > index.html << 'HTML'
<!DOCTYPE html>
<html><head><title>PP6 - Infrastructure as Code</title></head>
<body><h1>PP6 - Infrastructure as Code</h1><p>Servidor funcionando
correctamente</p></body></html>
HTML

cat > servidor.py << 'PYTHON'

```

```
import http.server
import socketserver
PORT = 3000
Handler = http.server.SimpleHTTPRequestHandler
with socketserver.TCPServer(("", PORT), Handler) as httpd:
    httpd.serve_forever()
PYTHON
chown -R ubuntu:ubuntu /home/ubuntu/pp6-web
nohup python3 /home/ubuntu/pp6-web/servidor.py > /tmp/server.log 2>&1 &
EOF
)
tags = {
    Name = "pp6-instancia-iac"
    Tipo = "Servidor-Aplicacion"
}
}
```

Guarda el archivo con **Ctrl+X**, luego **Y**, luego **Enter**.

Para verificar que se guardó correctamente:

```
wc -l main.tf
```

Salida esperada:

```
~171 main.tf
```

 **Explicación:** El archivo main.tf debe tener aproximadamente 250 líneas. Si tiene menos, verifica que hayas copiado todo el contenido correctamente.

Paso 2.4: Crear el archivo de outputs

Crea el archivo **outputs.tf**:
nano outputs.tf

 **Explicación:** El archivo **outputs.tf** define qué información queremos que Terraform nos muestre después del despliegue, como IPs públicas y URLs de acceso.

Agrega el siguiente contenido:

```

output "ip_publica_instancia" {
  description = "IP pública de la instancia EC2"
  value      = aws_instance.pp6_app.public_ip
}

output "url_aplicacion" {
  description = "URL completa para acceder a la aplicación"
  value      = "http://${aws_instance.pp6_app.public_ip}:3000"
}

output "comando_ssh" {
  description = "Comando para conectarse por SSH"
  value      = "ssh -i pp6-clave-ssh.pem
ubuntu@${aws_instance.pp6_app.public_ip}"
}

output "clave_privada_ssh" {
  description = "Clave privada SSH (MANTENER SECRETA)"
  value      = tls_private_key.pp6_key_pair.private_key_pem
  sensitive  = true
}

```

Guarda el archivo con **Ctrl+X**, luego **Y**, luego **Enter**.

Para verificar la estructura del proyecto:

```
ls -la
```

Salida esperada:

```

total 16
drwxrwxr-x 2 usuario usuario 4096 oct 22 14:30 .
drwxr-xr-x 3 usuario usuario 4096 oct 22 14:25 ..
-rw-rw-r-- 1 usuario usuario 8756 oct 22 14:30 main.tf
-rw-rw-r-- 1 usuario usuario  875 oct 22 14:28 outputs.tf
-rw-rw-r-- 1 usuario usuario  623 oct 22 14:27 variables.tf

```

💡 **Explicación:** Ahora tenemos los tres archivos básicos de Terraform: variables (configuración), main (recursos) y outputs (resultados). Esta es la estructura estándar de un proyecto de Infrastructure as Code.

Parte 3: Despliegue con Terraform

Paso 3.1: Inicialización

```
terraform init
```

Salida esperada:

```
Initializing the backend...
Initializing provider plugins...
- Finding hashicorp/aws versions matching "~> 5.0"...
- Finding hashicorp/tls versions matching "~> 4.0"...
- Installing hashicorp/aws v5.70.0...
- Installing hashicorp/tls v4.0.5...
Terraform has been successfully initialized!
```

 **Explicación:** `terraform init` descarga los proveedores necesarios e inicializa el directorio de trabajo.

Paso 3.2: Validación

```
terraform validate
```

Salida esperada:

```
Success! The configuration is valid.
```

Paso 3.3: Planificación

```
terraform plan
```

Salida esperada (resumida):

```
Terraform will perform the following actions:
# aws_instance.pp6_app will be created
```



```
+ resource "aws_instance" "pp6_app" {
  + ami          = "ami-0866a3c8686eaeeba"
  + instance_type = "t2.micro"
  ...
}
# aws_vpc.pp6_vpc will be created
+ resource "aws_vpc" "pp6_vpc" {
  + cidr_block = "10.0.0.0/16"
  ...
}
```

Plan: 9 to add, 0 to change, 0 to destroy.

Paso 3.4: Aplicación

```
terraform apply
```

Escribe **yes** cuando te pregunte si quieres continuar.

Salida esperada (al final):

Apply complete! Resources: 12 added, 0 changed, 0 destroyed.

Outputs:

comando_ssh = "ssh -i pp6-clave-ssh.pem ubuntu@54.123.45.67"

ip_publica_instancia = "54.123.45.67"

url_aplicacion = "http://54.123.45.67:3000"

Paso 3.5: Guardar la clave SSH

```
terraform output -raw clave_privada_ssh > pp6-clave-ssh.pem
chmod 600 pp6-clave-ssh.pem
```

 **Explicación:** Guardamos la clave privada generada automáticamente para poder acceder a la instancia.

Parte 4: Verificación del despliegue

Paso 4.1: Probar la aplicación

```
# Obtener la URL
URL=$(terraform output -raw url_aplicacion)
echo "URL a probar: $URL"
curl $URL
```

Importante: La aplicación puede tardar 5-10 minutos en estar disponible después del `terraform apply`.

Si recibes "Connection refused" - Pasos de Validación

Paso 1: Verificar que la instancia está corriendo

```
aws ec2 describe-instances --filters "Name=tag:Name,Values=pp6-instancia-iac"
--query 'Reservations[*].Instances[*].[InstanceId,State.Name,PublicIpAddress]'
--output table
```

Paso 2: Conectarse por SSH y verificar logs

```
ssh -i pp6-clave-ssh.pem ubuntu@$(terraform output -raw ip_publica_instancia)
```

Una vez conectado, ejecutar en orden:

```
# 1. Verificar proceso python
ps aux | grep python3

# 2. Verificar archivos creados
ls -la /home/ubuntu/pp6-web/
# Si el directorio no existe, crearlo manualmente:
mkdir -p /home/ubuntu/pp6-web
cd /home/ubuntu/pp6-web
echo '<html><head><title>PP6</title></head><body><h1>PP6
Funciona</h1></body></html>' > index.html
cat > servidor.py << 'EOF'
import http.server
import socketserver
PORT = 3000
Handler = http.server.SimpleHTTPRequestHandler
```

```
with socketserver.TCPServer(("", PORT), Handler) as httpd:
    httpd.serve_forever()
EOF
```

3. Iniciar servidor manualmente

```
nohup python3 servidor.py > /tmp/server.log 2>&1 &
```

4. Verificar que el puerto está abierto

```
ss -tlnp | grep 3000
```

5. Probar localmente

```
curl http://localhost:3000
```

Paso 3: Si no funciona, crear e iniciar manualmente

Crear todo desde cero

```
mkdir -p /home/ubuntu/pp6-web
```

```
cd /home/ubuntu/pp6-web
```

```
echo '<html><head><title>PP6</title></head><body><h1>PP6
Funciona</h1></body></html>' > index.html
```

```
cat > servidor.py << 'EOF'
```

```
import http.server
```

```
import socketserver
```

```
PORT = 3000
```

```
Handler = http.server.SimpleHTTPRequestHandler
```

```
with socketserver.TCPServer(("", PORT), Handler) as httpd:
```

```
    httpd.serve_forever()
```

```
EOF
```

```
nohup python3 servidor.py > /tmp/server.log 2>&1 &
```

```
sleep 5
```

```
curl http://localhost:3000
```

Paso 4: Salir y probar desde fuera

```
exit
```

```
curl $URL
```

Salida esperada: Una página HTML con el título "PP6 - Infrastructure as Code" y el mensaje "¡Éxito! Tu infraestructura en AWS está funcionando correctamente."

Paso 4.2: Verificar desde el navegador

También puedes abrir la URL en tu navegador web:

Mostrar la URL completa

`echo $URL`

Copia la URL y ábrela en tu navegador para ver la página web completa.

Paso 4.3: Acceso SSH

```
# Usar el comando SSH del output
```

```
ssh -i pp6-clave-ssh.pem ubuntu@54.123.45.67
```

Una vez conectado:

```
# Verificar el servicio web
```

```
sudo systemctl status pp6-web.service
```

Salida esperada:

- pp6-web.service - PP6 Web Server

```
Loaded: loaded (/etc/systemd/system/pp6-web.service; enabled; vendor preset: enabled)
```

```
Active: active (running) since Tue 2025-10-23 14:28:00 UTC; 5min ago
```

Para ver los logs:

```
sudo journalctl -u pp6-web.service -f
```

Salir de la instancia:

```
exit
```

Parte 5: Migración a OpenTofu

Paso 5.1: Crear directorio para OpenTofu

```
mkdir -p ../laboratorio-opentofu
```

```
cd ../laboratorio-opentofu
```

Paso 5.2: Copiar configuración

```
cp ../laboratorio-terraform/*.tf ./
```

Paso 5.3: Inicializar OpenTofu

```
tofu init
```

Salida esperada:

```
Initializing the backend...
Initializing provider plugins...
- Finding hashicorp/aws versions matching "~> 5.0"...
- Installing hashicorp/aws v5.70.0...
OpenTofu has been successfully initialized!
```

Paso 5.4: Importar estado existente

```
cp ../laboratorio-terraform/terraform.tfstate ./terraform.tfstate
```

Paso 5.5: Verificar con OpenTofu

```
tofu plan
```

Salida esperada:

```
No changes. Your infrastructure matches the configuration.
```

💡 **Explicación:** OpenTofu reconoce la infraestructura creada por Terraform, demostrando compatibilidad total.

Parte 6: Limpieza

Paso 6.1: Destruir recursos

```
tofu destroy
```

Escribe **yes** cuando te pregunte

Salida esperada:

Destroy complete! Resources: 9 destroyed

Paso 6.2: Verificar limpieza

```
aws ec2 describe-instances --query  
'Reservations[*].Instances[?State.Name==`running`].[InstanceId,State.Name]'  
--output table
```

Salida esperada:

|DescribeInstances|

¡Felicitaciones! Has completado el laboratorio de Infrastructure as Code.

Importante: Siempre ejecuta **destroy** para evitar costos en AWS.

Bibliografía

- [Terraform Documentation](#)
- [OpenTofu Documentation](#)
- [AWS Free Tier](#)